

Task 16: Assignment 16

Name of student: Prachi Purushottam Rudraksha

Domain/subject name: AI&ML

Collage and Branch name: T.B.Girwalkar Polytechnic Ambajogai (Electronics & Computer TE)

Q1. Load the insurance dataset using pandas and display the first 10 rows of the data.

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) insurance.xlsx
insurance.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 49840 bytes, last modified: 7/8/2026 - 100% done
Saving insurance.xlsx to insurance.xlsx

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_excel("insurance.xlsx")
df.head(10)
```

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86
5	31	female	25.7	0	no	southeast	3756.62
6	46	female	33.4	1	no	southeast	8240.59
7	37	female	27.7	3	no	northwest	7281.51
8	37	male	29.8	2	no	northeast	6406.41
9	60	female	25.8	0	no	northwest	28923.14

Next steps: [Generate code with df](#) [New interactive sheet](#)

Q2. Use pandas functions to get basic information about the dataset:

. Shape of the data (shape)

* Column names and data types (info())

◦ Statistical summary of numerical columns (describe())

```
print("First 10 Rows of Dataset")
print(df.head(10))
```

... First 10 Rows of Dataset

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86
5	31	female	25.7	0	no	southeast	3756.62
6	46	female	33.4	1	no	southeast	8240.59
7	37	female	27.7	3	no	northwest	7281.51
8	37	male	29.8	2	no	northeast	6406.41
9	60	female	25.8	0	no	northwest	28923.14

Q3. Check if the dataset contains any nul/missing values. Show the count of null values for each column.

```
print("\nShape of Dataset:")
print(df.shape)

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())
```

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   expenses    1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None

```

```

Statistical Summary:

```

	age	bmi	children	expenses
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.665471	1.094918	13270.422414
std	14.049960	6.098382	1.205493	12110.011240
min	18.000000	16.000000	0.000000	1121.870000
25%	27.000000	26.300000	0.000000	4740.287500
50%	39.000000	30.400000	1.000000	9382.030000
75%	51.000000	34.700000	2.000000	16639.915000
max	64.000000	53.100000	5.000000	63770.430000

Q4. Identify numerical and categorical columns in the dataset. Print them separately.

```

> print("\nMissing Values:")
print(df.isnull().sum())

..
Missing Values:
age         0
sex         0
bmi         0
children    0
smoker      0
region      0
expenses    0
dtype: int64

```

Q5. Create distribution plots for all numerical columns (age, bmi, children, charges):

- * Use `sns.histplot()` or `sns.distplot()`
- . Create one plot per column (total 4 plots)
- Add meaningful titles and labels

```
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

categorical_cols = df.select_dtypes(include=['object']).columns

print("\nNumerical Columns:")
print(list(numerical_cols))

print("\nCategorical Columns:")
print(list(categorical_cols))

...
Numerical Columns:
['age', 'bmi', 'children', 'expenses']

Categorical Columns:
['sex', 'smoker', 'region']
```

Q6. Create count plots for all categorical columns (sex, smoker, region):

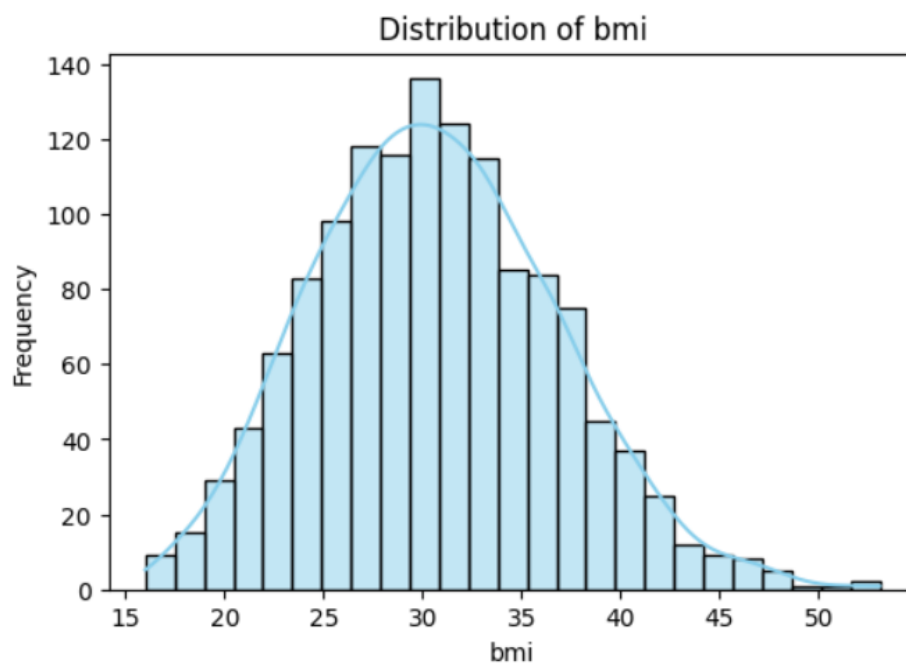
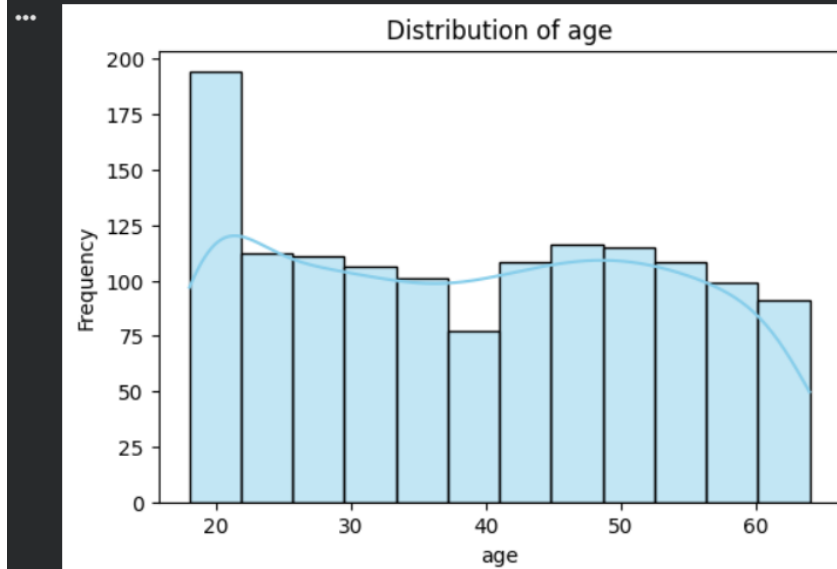
- * Use `sns.countplot()` for each column

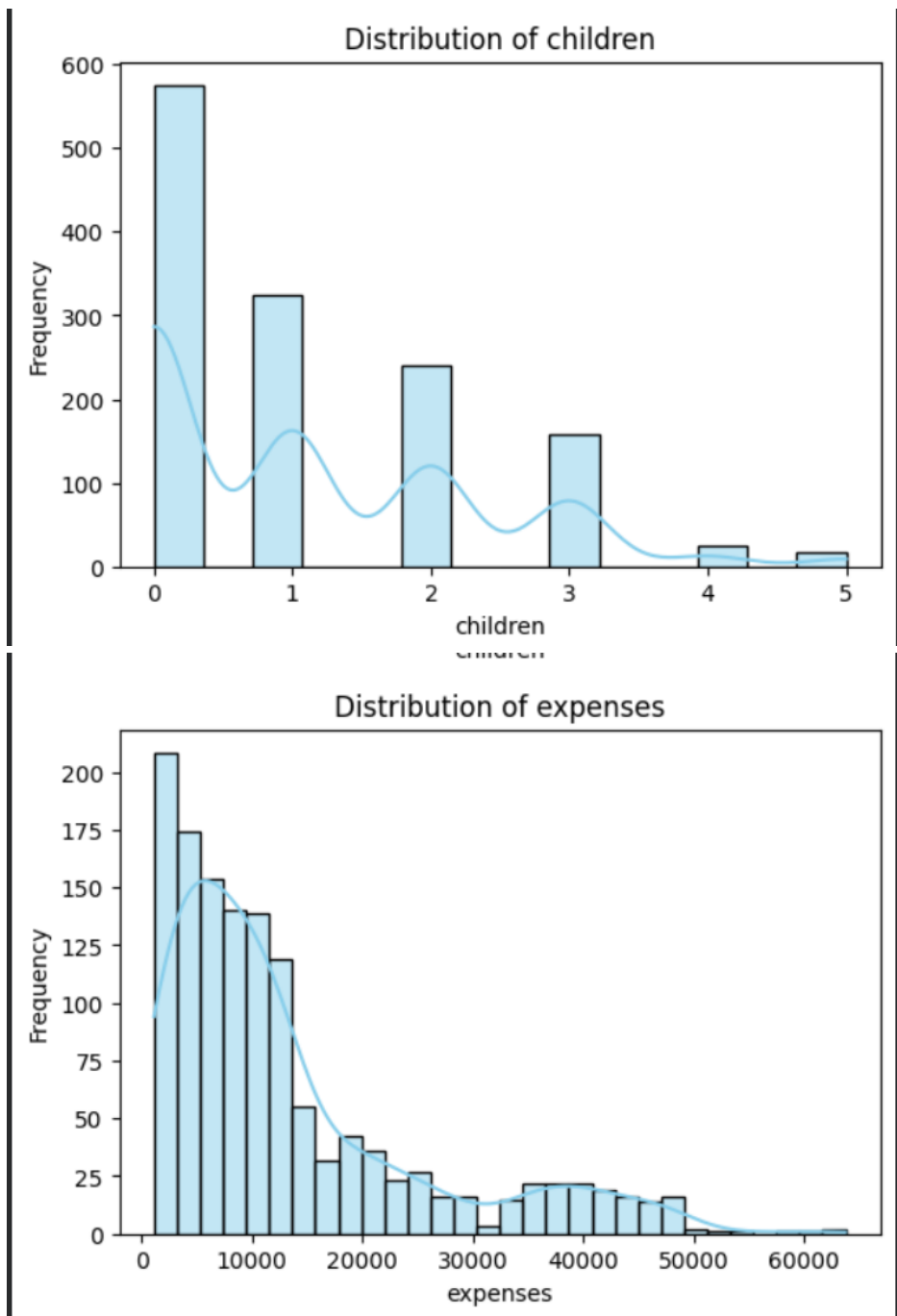
- . Add titles and labels for clarity

```

for col in numerical_cols:
    plt.figure(figsize=(6,4))
    sns.histplot(df[col], kde=True, color='skyblue')
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.show()

```





Q7. Create a heatmap to show the correlation between numerical variables.

- Use `sns.heatmap()`

- * Annotate the values and use a suitable color palette

```

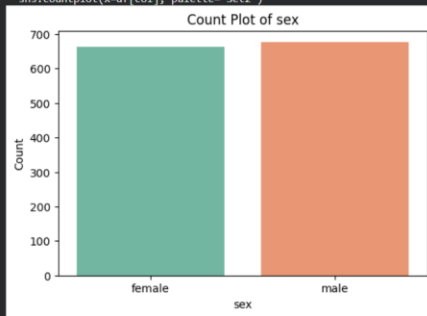
for col in categorical_cols:
    plt.figure(figsize=(6,4))
    sns.countplot(x=df[col], palette='Set2')
    plt.title(f"Count Plot of {col}")
    plt.xlabel(col)
    plt.ylabel("Count")
    plt.show()

```

```

... /tmp/ipykernel_1282/259724086.py:13: FutureWarning:
    Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.
    sns.countplot(x=df[col], palette='Set2')

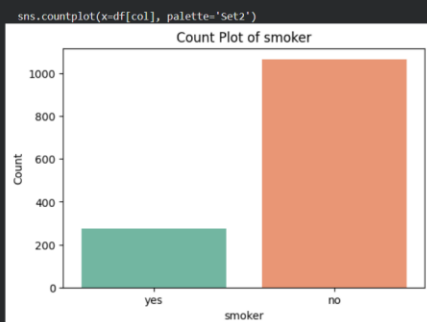
```



```

... /tmp/ipykernel_1282/259724086.py:13: FutureWarning:
    Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.
    sns.countplot(x=df[col], palette='Set2')

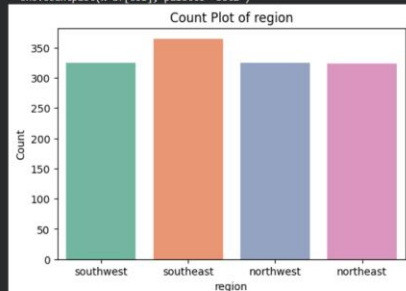
```



```

... /tmp/ipykernel_1282/259724086.py:13: FutureWarning:
    Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.
    sns.countplot(x=df[col], palette='Set2')

```



Q8. Perform the following analysis on the charges column:. Find the average insurance charges

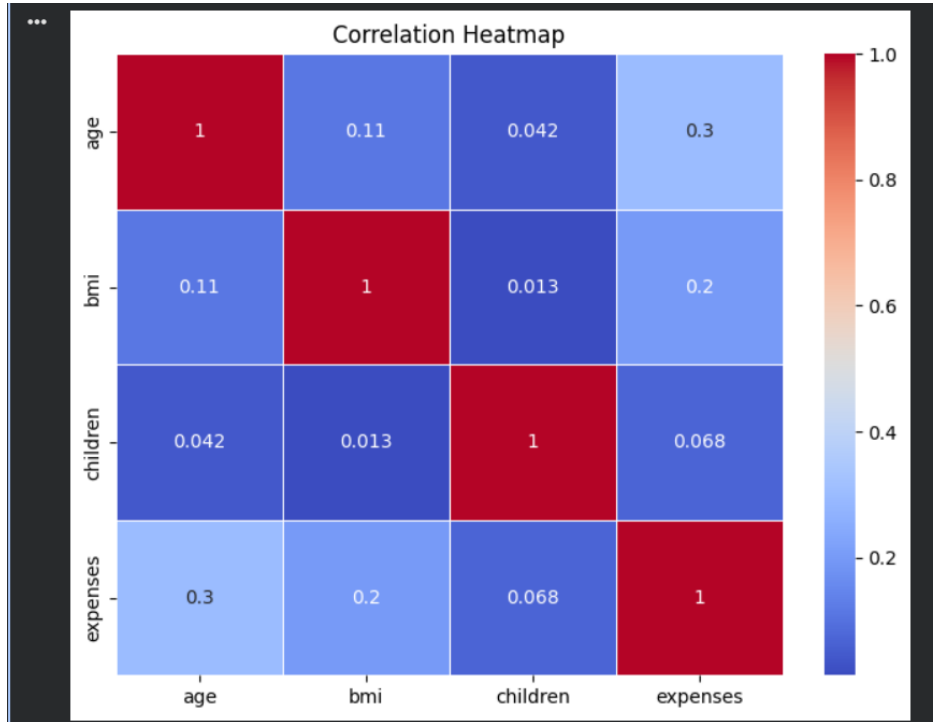
* Find the maximum and minimum charges

* Group by smoker and compare the average charges for smokers vs non-Smokers.

```
plt.figure(figsize=(8,6))

sns.heatmap(df[numerical_cols].corr(),
            annot=True,
            cmap='coolwarm',
            linewidths=0.5)

plt.title("Correlation Heatmap")
plt.show()
```

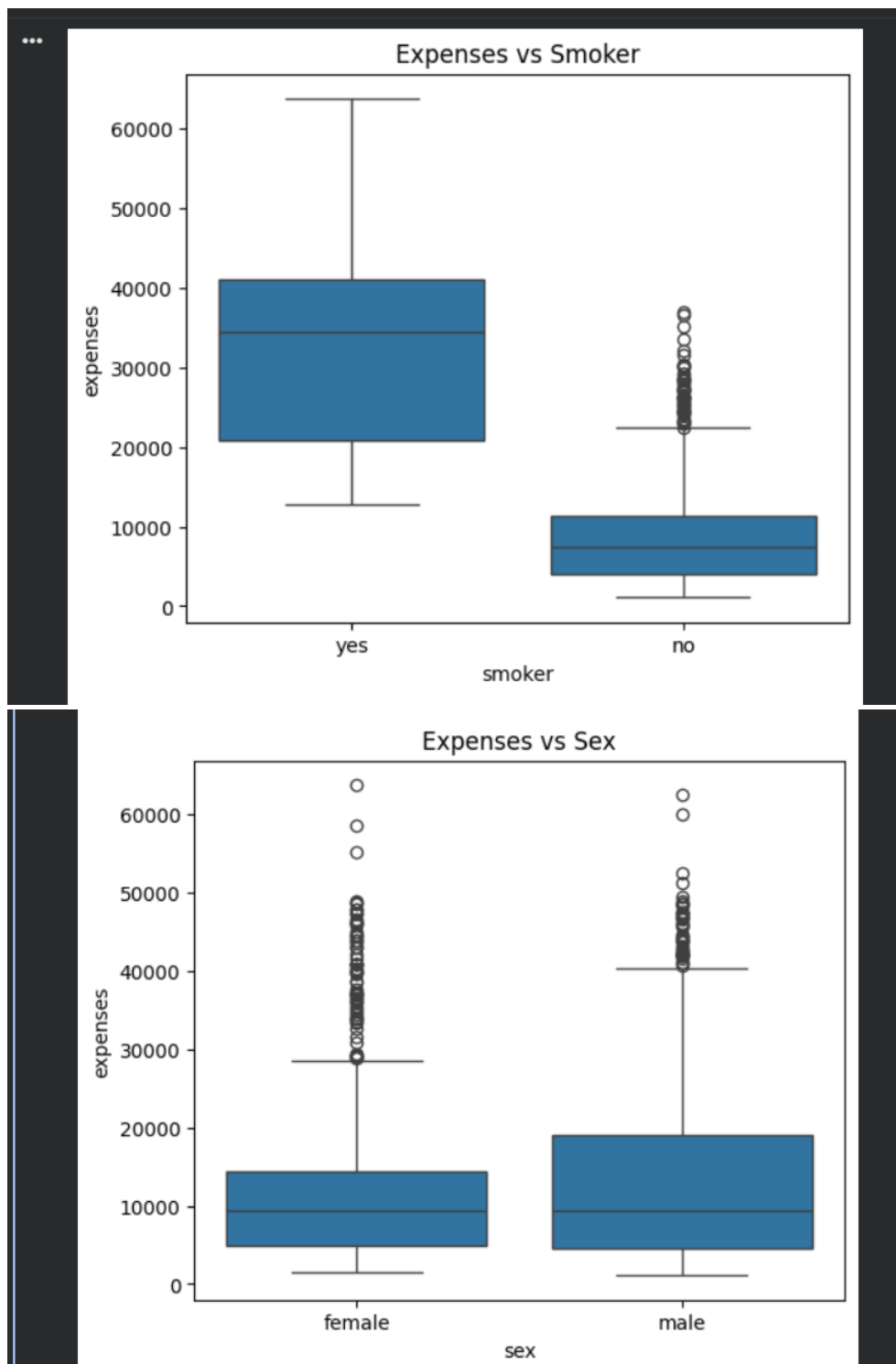


Q9. Create a boxplot to show the distribution of charges with respect to smoker and sex.

(You can create two separate boxplots or use hue parameter)

```
plt.figure(figsize=(6,5))
sns.boxplot(x='smoker', y='expenses', data=df)
plt.title("Expenses vs Smoker")
plt.show()

plt.figure(figsize=(6,5))
sns.boxplot(x='sex', y='expenses', data=df)
plt.title("Expenses vs Sex")
plt.show()
```

Q10.(Mini Project / Analysis Summary

Write a short summary (in comments or markdown) answering the following:1. What is the average age and BMI of the customers?

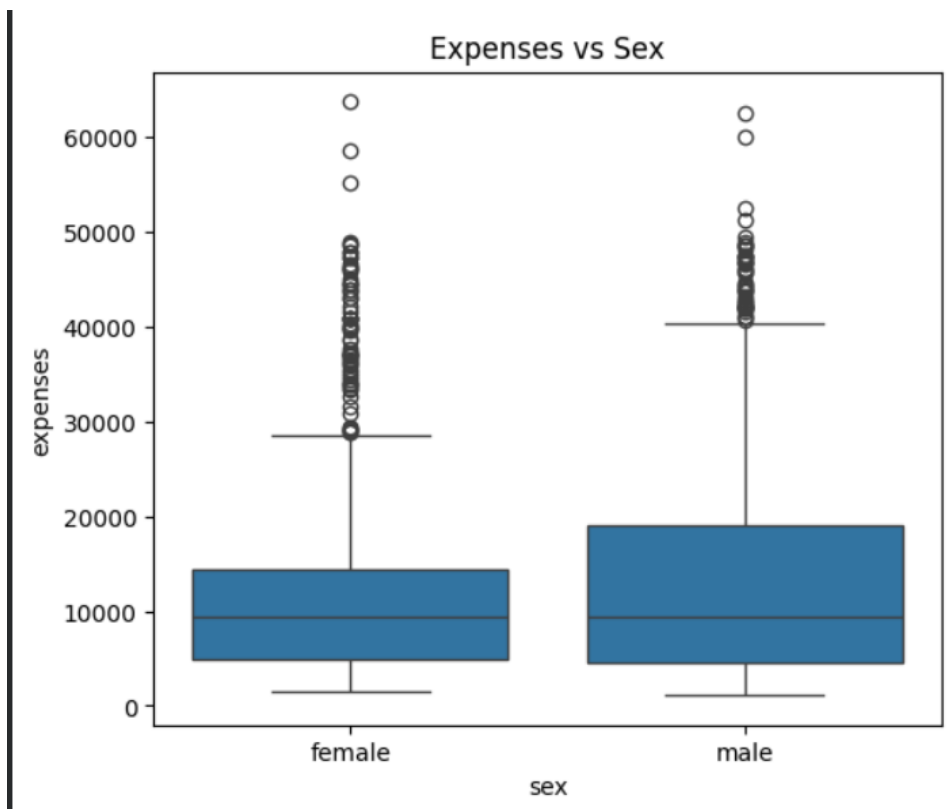
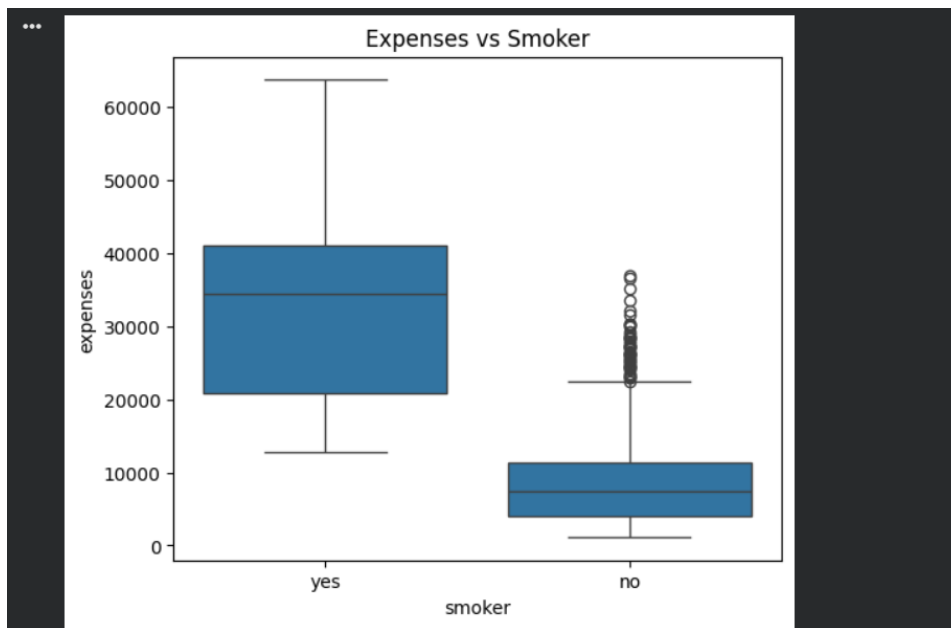
2. How does smoking affect insurance charges?

3. Which region has the highest number of customers?

4. Any interesting observations from the visualizations?

```
plt.figure(figsize=(6,5))
sns.boxplot(x='smoker', y='expenses', data=df)
plt.title("Expenses vs Smoker")
plt.show()

plt.figure(figsize=(6,5))
sns.boxplot(x='sex', y='expenses', data=df)
plt.title("Expenses vs Sex")
plt.show()
```



Optional Task

Find dataset by yourself and perform EDA on it.

(Optional task carries bonus marks)

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset
df = sns.load_dataset("titanic")

# Display first rows
print("First 5 Rows:")
print(df.head())

# Shape of dataset
print("\nShape of Dataset:")
print(df.shape)

# Information
print("\nDataset Information:")
df.info()

# Statistical Summary
print("\nStatistical Summary:")
print(df.describe())

# Missing Values
print("\nMissing Values:")
print(df.isnull().sum())

# Numerical and Categorical Columns
print("\nNumerical Columns:")
print(df.select_dtypes(include=['int64', 'float64']).columns)
```

```
print("\nCategorical Columns:")
print(df.select_dtypes(include=['object', 'category', 'bool']).columns)

# Histogram - Numerical Data
df[['age', 'fare']].hist(figsize=(8,4))
plt.suptitle("Distribution of Age and Fare")
plt.show()

# Count Plot - Survival
sns.countplot(x='survived', data=df)
plt.title("Survival Count")
plt.xlabel("Survived")
plt.ylabel("Count")
plt.show()

# Count Plot - Gender Survival
sns.countplot(x='sex', hue='survived', data=df)
plt.title("Survival by Gender")
plt.show()

# Correlation Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(
    df.select_dtypes(include=['int64', 'float64']).corr(),
    annot=True,
    cmap="coolwarm"
)
```

```

plt.title("Correlation Heatmap")
plt.show()

# Boxplot
sns.boxplot(x='class', y='fare', data=df)
plt.title("Fare Distribution by Passenger Class")
plt.show()

# Analysis Summary
print("\n--- EDA Summary ---")
print("Average Age:", df['age'].mean())
print("Average Fare:", df['fare'].mean())

print("\nPassenger Class Count:")
print(df['class'].value_counts())

print("\nObservation:")
print("- Females had higher survival chances than males.")
print("- First class passengers paid higher fares.")
print("- Third class had the highest number of passengers.")
print("- Dataset contains missing values in some columns.")

... First 5 Rows:
   survived  pclass    sex  age  sibsp  parch    fare embarked  class \
0         0      3  male  22.0     1     0   7.2500         S   Third
1         1      1  female 38.0     1     0  71.2833         C   First
2         1      3  female 26.0     0     0   7.9250         S   Third
3         1      1  female 35.0     1     0  53.1000         S   First
4         0      3   male  35.0     0     0   8.0500         S   Third

```

```

...   who  adult_male  deck  embark_town  alive  alone
0   man          True  NaN  Southampton    no   False
1 woman         False   C    Cherbourg   yes   False
2 woman         False  NaN  Southampton   yes    True
3 woman         False   C    Southampton   yes   False
4   man          True  NaN  Southampton    no    True

Shape of Dataset:
(891, 15)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
8   class       891 non-null    category
9   who         891 non-null    object
10  adult_male   891 non-null    bool
11  deck         203 non-null    category
12  embark_town  889 non-null    object
13  alive        891 non-null    object
14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB

```

```

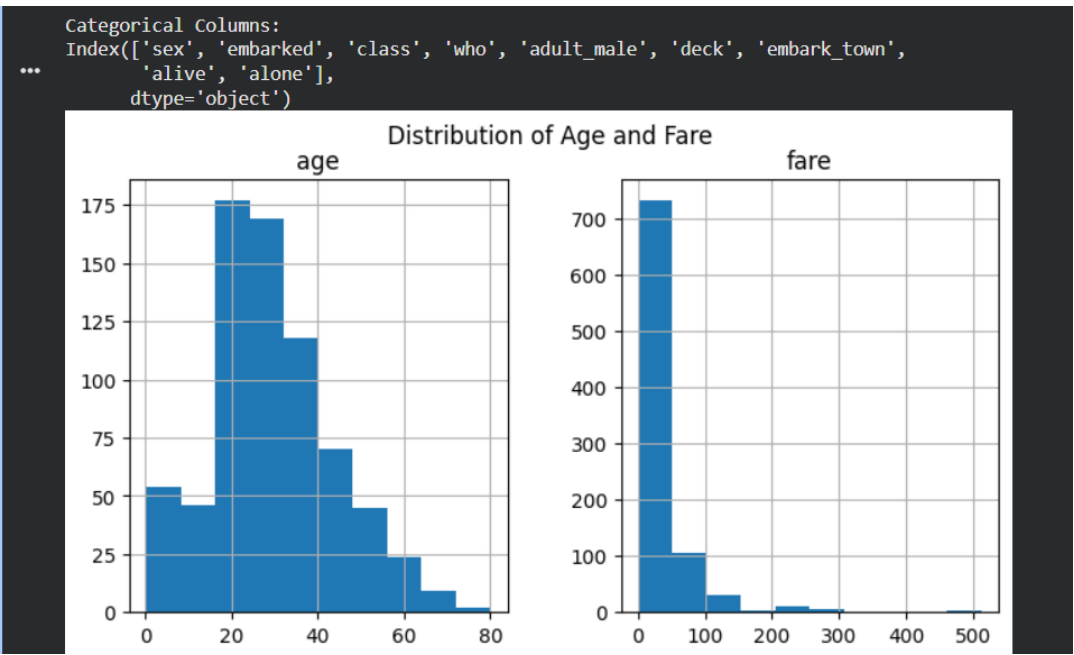
Statistical Summary:
...
      survived      pclass      age      sibsp      parch      fare
count  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean    0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std     0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min     0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%     0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%     0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%     1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max     1.000000    3.000000   80.000000    8.000000    6.000000  512.329200

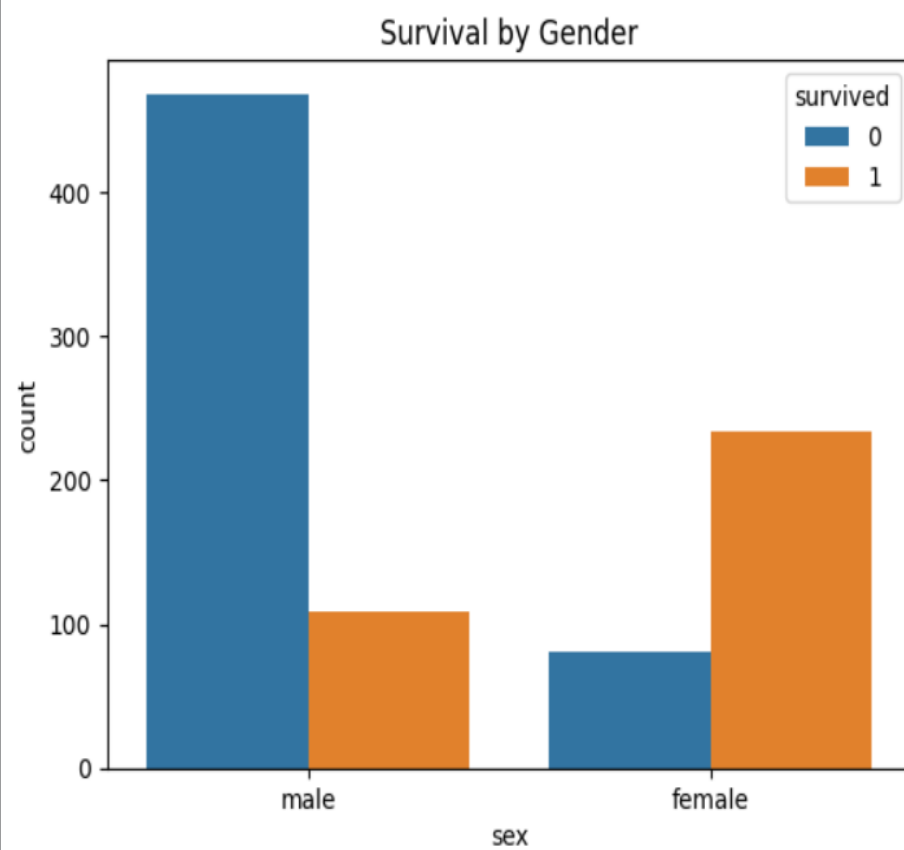
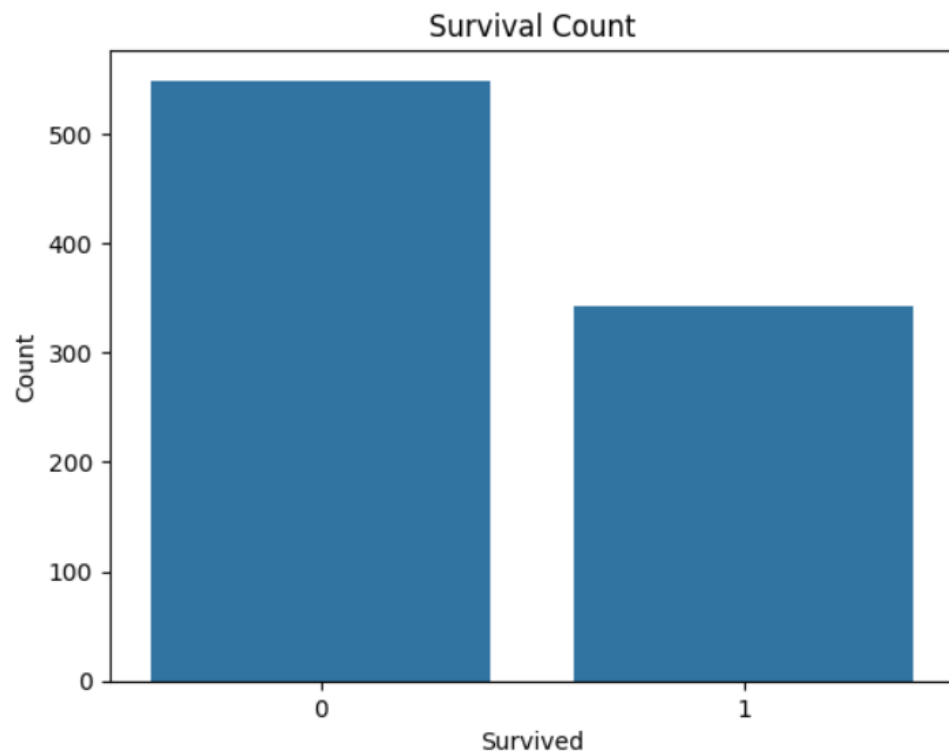
Missing Values:
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64

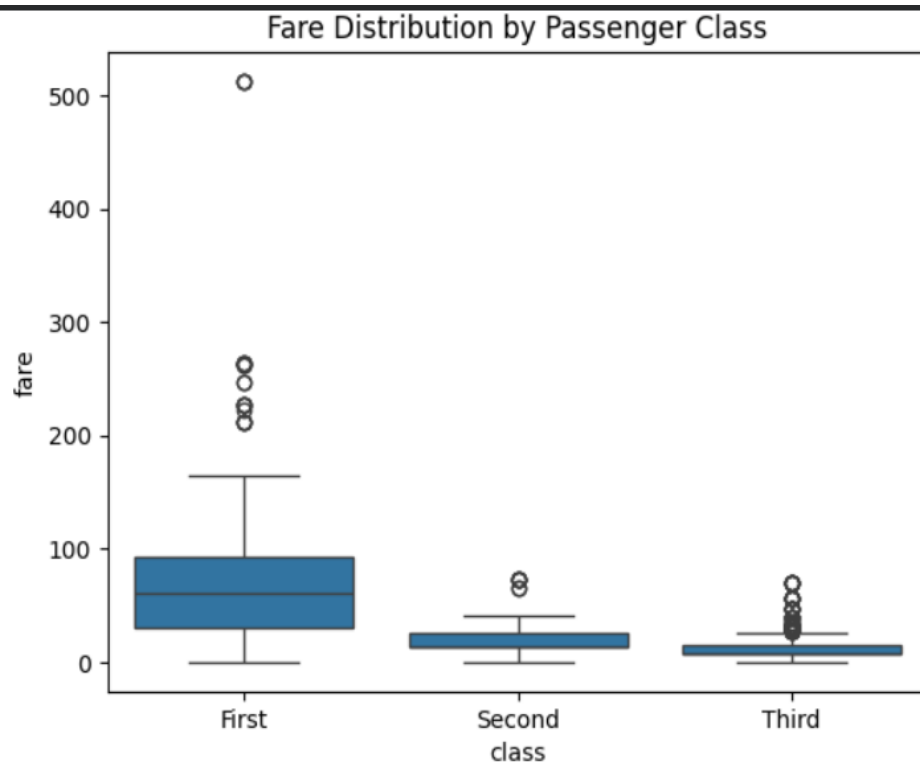
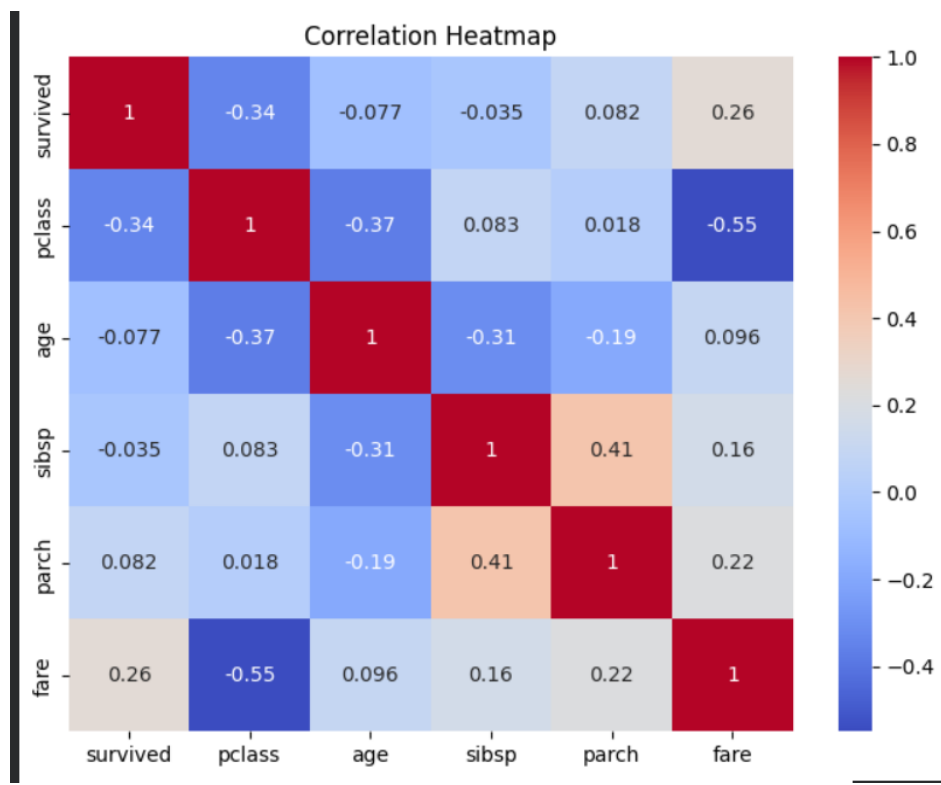
Numerical Columns:
Index(['survived', 'pclass', 'age', 'sibsp', 'parch', 'fare'], dtype='object')

Categorical Columns:
Index(['sex', 'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
      'alive', 'alone'],
      dtype='object')

```







```
--- EDA Summary ---  
Average Age: 29.69911764705882  
Average Fare: 32.204207968574636
```

Passenger Class Count:

class

Third 491

First 216

Second 184

Name: count, dtype: int64

Observation:

- Females had higher survival chances than males.
- First class passengers paid higher fares.
- Third class had the highest number of passengers.
- Dataset contains missing values in some columns.